UD5. Programación de bases de datos

Bloque 3. Estruturas de control de fluxo

Neste bloque estudarase como incorporar lóxica de programación aos guións SQL mediante estruturas condicionais e repetitivas. Estas estruturas permiten que os procedementos e funcións de MariaDB tomen decisións, executen instrucións diferentes segundo determinadas condicións e repitan operacións automaticamente.

1. Introdución ao SQL procedemental

1.1. SQL declarativo e SQL procedemental

SQL é unha linguaxe principalmente declarativa: indicamos que información queremos consultar ou modificar, pero non describimos paso a paso como debe realizarse a operación.

O SQL procedemental incorpora elementos propios das linguaxes de programación, como variables, condicións, bucles e bloques de instrucións. Isto permite definir unha secuencia de operacións que o servidor executará seguindo unha lóxica determinada.

En MariaDB, estas estruturas utilízanse principalmente dentro de procedementos almacenados, funcións, disparadores e eventos.

1.2. Bloques BEGIN ... END

A estrutura `BEGIN ... END` permite agrupar varias instrucións nun único bloque. As instrucións execútanse secuencialmente, na orde na que aparecen.

```
BEGIN
    -- Primeira instrución
    SET @numero = 10;

    -- Segunda instrución
    SET @resultado = @numero * 2;

    -- Terceira instrución
    SELECT @resultado;
END
```

Os bloques poden conter variables locais, instrucións condicionais, bucles e outros bloques. Son a base da programación almacenada en MariaDB.

Importante: BEGIN ... END e os delimitadores

Cando se crea un procedemento ou unha función que contén varias instrucións, é necesario modificar temporalmente o delimitador utilizado polo cliente SQL. Deste xeito, o punto e coma pode seguir utilizándose para separar instrucións dentro do bloque.

En MySQL Workbench e na consola `mariadb` pódese utilizar `DELIMITER` . Esta instrución é unha directiva do cliente, non unha instrución SQL executada polo servidor.

Exemplo: primeiro procedemento

```
DELIMITER //

CREATE PROCEDURE exemplo_bloque()
BEGIN
    DECLARE numero INT DEFAULT 10;
    DECLARE resultado INT;

    SET resultado = numero * 2;

    SELECT resultado AS resultado_calculado;
END //

DELIMITER ;

-- Executar o procedemento
CALL exemplo_bloque();
```

O procedemento declara dúas variables locais, realiza unha operación matemática e mostra o resultado mediante unha consulta.

1.3. Declaración de variables locais

As variables locais permiten almacenar valores temporais dentro dun bloque de programación. Decláranse mediante `DECLARE` e só están dispoñibles dentro do bloque no que se definiron.

```
DECLARE nome VARCHAR(50);
DECLARE idade INT DEFAULT 18;
DECLARE total DECIMAL(10,2) DEFAULT 0;
```

As declaracións deben aparecer ao principio do bloque `BEGIN ... END`, antes das instrucións executables.

Para asignar valores utilízase a instrución `SET`.

```
SET nome = 'Ana';
SET idade = 25;
SET total = 150.50;
```

> **Variables locais e variables de usuario**
> Non se deben confundir as variables locais declaradas con `DECLARE` coas variables de usuario que comezan por `@`. As primeiras pertencen ao bloque procedemental; as segundas pertencen á conexión actual.

2. Estruturas condicionais IF

2.1. Concepto

As estruturas condicionais permiten executar diferentes instrucións en función de que se cumpra ou non unha condición. Son fundamentais para incorporar capacidade de decisión aos procedementos almacenados.

En MariaDB, a estrutura `IF ... THEN` permite avaliar unha expresión lóxica. Se a condición é verdadeira, execútase o bloque asociado.

2.2. Estrutura IF ... THEN ... END IF

É a forma máis sinxela dunha estrutura condicional.

```
IF condicion THEN
    instrucions;
END IF;
```

Exemplo: comprobar unha nota

```
DELIMITER //

CREATE PROCEDURE comprobar_nota(IN p_nota DECIMAL(4,2))
BEGIN
    IF p_nota >= 5 THEN
        SELECT 'Aprobado' AS resultado;
    END IF;
END //

DELIMITER ;

CALL comprobar_nota(7.5);
```

Se a nota recibida é igual ou superior a 5, o procedemento mostra a mensaxe «Aprobado». Se non se cumpre a condición, non executa ningunha instrución dentro do bloque.

2.3. Estrutura IF ... THEN ... ELSE ... END IF

A cláusula `ELSE` permite definir un bloque alternativo que se executará cando a condición principal sexa falsa.

```
IF condicion THEN
    instrucions_se_verdadeiro;
ELSE
    instrucions_se_falso;
END IF;
```

Exemplo: aprobado ou suspenso

```
DELIMITER //

CREATE PROCEDURE resultado_alumno(IN p_nota DECIMAL(4,2))
BEGIN
    IF p_nota >= 5 THEN
        SELECT 'Aprobado' AS resultado;
    ELSE
        SELECT 'Suspenso' AS resultado;
    END IF;
END //

DELIMITER ;

CALL resultado_alumno(4.5);
```

2.4. Estrutura IF ... ELSEIF ... ELSE

Cando é necesario comprobar varias condicións, pódese utilizar `ELSEIF`. As condicións avalíanse na orde na que aparecen e execútase o primeiro bloque cuxa condición sexa verdadeira.

```
IF condicion1 THEN
    instrucions1;
ELSEIF condicion2 THEN
    instrucions2;
ELSEIF condicion3 THEN
    instrucions3;
ELSE
    instrucions_por_defecto;
END IF;
```

Exemplo: cualificación dunha nota

```
DELIMITER //

CREATE PROCEDURE cualificar_nota(IN p_nota DECIMAL(4,2))
BEGIN
    IF p_nota >= 9 THEN
        SELECT 'Sobresaliente' AS cualificacion;
    ELSEIF p_nota >= 7 THEN
        SELECT 'Notable' AS cualificacion;
    ELSEIF p_nota >= 5 THEN
        SELECT 'Aprobado' AS cualificacion;
    ELSE
        SELECT 'Suspenso' AS cualificacion;
    END IF;
END //

DELIMITER ;

CALL cualificar_nota(8.25);
```

2.5. Condicións compostas

As condicións poden combinarse mediante operadores lóxicos para avaliar máis dun requisito.

- `AND` : todas as condicións deben ser verdadeiras.
- `OR` : polo menos unha condición debe ser verdadeira.
- `NOT` : inverte o resultado dunha condición.

Exemplo: comprobar varios requisitos

```
DELIMITER //

CREATE PROCEDURE comprobar_acceso(
    IN p_idade INT,
    IN p_activo BOOLEAN
)
BEGIN
    IF p_idade >= 18 AND p_activo = TRUE THEN
        SELECT 'Acceso permitido' AS resultado;
    ELSE
        SELECT 'Acceso denegado' AS resultado;
    END IF;
END //

DELIMITER ;

CALL comprobar_acceso(20, TRUE);
```

3. Estrutura de selección múltiple CASE

3.1. Concepto

A estrutura `CASE` permite seleccionar unha opción entre varias alternativas. É especialmente útil cando existen múltiples valores ou condicións que producen resultados diferentes.

Dentro dun procedemento ou función, `CASE` pode utilizarse como instrución procedemental. Non debe confundirse coa expresión `CASE` que tamén se pode empregar directamente nunha consulta SQL.

3.2. CASE simple

A modalidade simple compara unha expresión cunha serie de valores. Cando se produce unha coincidencia, execútase o bloque correspondente.

```
CASE expresion
    WHEN valor1 THEN
        instrucions1;
    WHEN valor2 THEN
        instrucions2;
    WHEN valor3 THEN
        instrucions3;
    ELSE
        instrucions_por_defecto;
END CASE;
```

Exemplo: identificar un día da semana

```
DELIMITER //

CREATE PROCEDURE nome_dia(IN p_dia INT)
BEGIN
    CASE p_dia
        WHEN 1 THEN
            SELECT 'Luns' AS dia;
        WHEN 2 THEN
            SELECT 'Martes' AS dia;
        WHEN 3 THEN
            SELECT 'Mércores' AS dia;
        WHEN 4 THEN
            SELECT 'Xoves' AS dia;
        WHEN 5 THEN
            SELECT 'Venres' AS dia;
        WHEN 6 THEN
            SELECT 'Sábado' AS dia;
        WHEN 7 THEN
            SELECT 'Domingo' AS dia;
        ELSE
            SELECT 'Día non válido' AS dia;
    END CASE;
END //

DELIMITER ;

CALL nome_dia(3);
```

3.3. CASE buscado

A modalidade buscada avalía unha serie de condicións lóxicas. Execútase o primeiro bloque cuxa condición sexa verdadeira.

```
CASE
    WHEN condicion1 THEN
        instrucions1;
    WHEN condicion2 THEN
        instrucions2;
    ELSE
        instrucions_por_defecto;
END CASE;
```

Exemplo: clasificar un produto polo prezo

```
DELIMITER //

CREATE PROCEDURE clasificar_produto(IN p_prezo DECIMAL(10,2))
BEGIN
    CASE
        WHEN p_prezo < 10 THEN
            SELECT 'Barato' AS categoria;
        WHEN p_prezo < 50 THEN
            SELECT 'Prezo medio' AS categoria;
        WHEN p_prezo < 100 THEN
            SELECT 'Caro' AS categoria;
        ELSE
            SELECT 'Moi caro' AS categoria;
    END CASE;
END //

DELIMITER ;

CALL clasificar_produto(65);
```

3.4. Diferenzas entre IF e CASE

Estrutura	Uso habitual
`IF`	Comprobar unha condición ou varias condicións alternativas.
`CASE` simple	Comparar unha expresión con distintos valores posibles.
`CASE` buscado	Avaliar varias condicións lóxicas.

> **Importante: instrución CASE e expresión CASE**
> A instrución `CASE ... END CASE` permite executar bloques de instrucións dentro dun programa almacenado. A expresión `CASE ... END` devolve un valor e pode utilizarse directamente en consultas SQL.
>
> Non se debe engadir `END CASE` cando se utiliza a expresión CASE dentro dun `SELECT` .

4. Estruturas repetitivas

4.1. Concepto

As estruturas repetitivas, tamén chamadas bucles, permiten executar varias veces un conxunto de instrucións. A repetición pode depender dunha condición ou dun criterio de finalización.

MariaDB dispón de tres estruturas principais:

- `WHILE` .
- `REPEAT ... UNTIL` .
- `LOOP` .

4.2. Bucle WHILE

O bucle `WHILE` executa as instrucións mentres se cumpra unha condición. A condición compróbase antes de cada iteración.

```
WHILE condicion DO
    instrucions;
END WHILE;
```

Se a condición é falsa desde o principio, o bloque non se executa ningunha vez.

Exemplo: contar do 1 ao 5

```
DELIMITER //

CREATE PROCEDURE contar_while()
BEGIN
    DECLARE contador INT DEFAULT 1;

    WHILE contador <= 5 DO
        SELECT contador;
        SET contador = contador + 1;
    END WHILE;
END //

DELIMITER ;

CALL contar_while();
```

O procedemento inicializa un contador en 1. En cada iteración mostra o seu valor e increméntao. O bucle remata cando o contador supera o valor 5.

4.3. Bucle REPEAT ... UNTIL

O bucle `REPEAT` executa as instrucións e comproba a condición ao final de cada iteración. Repítese ata que a condición indicada en `UNTIL` sexa verdadeira.

```
REPEAT
    instrucions;
UNTIL condicion
END REPEAT;
```

A diferenza de `WHILE`, o bloque execútase polo menos unha vez, xa que a condición se comproba ao final.

Exemplo: contar do 1 ao 5

```
DELIMITER //

CREATE PROCEDURE contar_repeat()
BEGIN
    DECLARE contador INT DEFAULT 1;

    REPEAT
        SELECT contador;
        SET contador = contador + 1;
    UNTIL contador > 5
    END REPEAT;
END //

DELIMITER ;

CALL contar_repeat();
```

4.4. Bucle LOOP

O bucle `LOOP` permite repetir un bloque de instrucións sen establecer unha condición directamente na súa cabeceira. A saída debe controlarse mediante instrucións como `LEAVE`.

```
nome_bucle: LOOP
    instrucions;

    IF condicion_saida THEN
        LEAVE nome_bucle;
    END IF;
END LOOP nome_bucle;
```

Exemplo: contar do 1 ao 5 con LOOP

```
DELIMITER //

CREATE PROCEDURE contar_loop()
BEGIN
    DECLARE contador INT DEFAULT 1;

    bucle: LOOP
        SELECT contador;

        SET contador = contador + 1;

        IF contador > 5 THEN
            LEAVE bucle;
        END IF;
    END LOOP bucle;
END //

DELIMITER ;

CALL contar_loop();
```

4.5. Comparación das estruturas repetitivas

Estrutura	Momento de avaliación	Execución mínima	Finalización
`WHILE`	Antes de cada iteración.	Zero veces.	Cando a condición deixa de cumprirse.
`REPEAT`	Ao final de cada iteración.	Unha vez.	Cando a condición de UNTIL é verdadeira.
`LOOP`	Non ten condición propia.	Depende da saída.	Mediante LEAVE ou outra saída do bloque.

Evitar os bucles infinitos
É fundamental que un bucle teña unha condición de saída alcanzable. Se a variable utilizada na condición non cambia correctamente, o programa pode quedar executándose indefinidamente.

5. Control das iteracións: LEAVE e ITERATE

5.1. Instrución LEAVE

A instrución `LEAVE` permite abandonar inmediatamente un bucle ou un bloque etiquetado. Utilízase cando se cumpre unha condición que fai innecesario continuar executando as iteracións.

```
nome_bucle: LOOP
    instrucions;

    IF condicion_saida THEN
        LEAVE nome_bucle;
    END IF;
END LOOP nome_bucle;
```

Exemplo: finalizar un bucle ao chegar a 10

```
DELIMITER //

CREATE PROCEDURE finalizar_contador()
BEGIN
    DECLARE contador INT DEFAULT 1;

    bucle: LOOP
        SELECT contador;

        IF contador = 10 THEN
            LEAVE bucle;
        END IF;

        SET contador = contador + 1;
    END LOOP bucle;
END //

DELIMITER ;

CALL finalizar_contador();
```

5.2. Instrución ITERATE

A instrución `ITERATE` permite saltar directamente á seguinte iteración dun bucle, sen executar as instrucións restantes da iteración actual.

```
nome_bucle: LOOP
    instrucions;

    IF condicion_saltar THEN
        ITERATE nome_bucle;
    END IF;

    outras_instrucions;
END LOOP nome_bucle;
```

Exemplo: mostrar só os números impares

```
DELIMITER //

CREATE PROCEDURE mostrar_impares()
BEGIN
    DECLARE contador INT DEFAULT 1;

    bucle: WHILE contador <= 10 DO

        IF MOD(contador, 2) = 0 THEN
            SET contador = contador + 1;
            ITERATE bucle;
        END IF;

        SELECT contador AS numero_impar;

        SET contador = contador + 1;

    END WHILE bucle;
END //

DELIMITER ;

CALL mostrar_impares();
```

Cando o contador é par, increméntase e execútase `ITERATE`, polo que se salta a instrución `SELECT`. Os números impares móstranse normalmente.

5.3. Diferenzas entre LEAVE e ITERATE

Instrución	Comportamento
`LEAVE`	Abandona completamente o bucle ou bloque etiquetado.
`ITERATE`	Salta ao inicio da seguinte iteración do bucle.

Ambas as instrucións utilizan unha etiqueta para identificar o bloque ao que fan referencia. Isto é especialmente útil cando existen bucles aniñados.

6. Combinación de condicións e bucles

As estruturas condicionais e repetitivas poden combinarse dentro dun mesmo procedemento. Deste xeito é posible realizar tarefas máis complexas, como percorrer unha serie de valores, comprobar condicións e executar operacións diferentes segundo os resultados.

Exemplo: sumar os números pares dunha serie

```
DELIMITER //

CREATE PROCEDURE sumar_pares(IN p_limite INT)
BEGIN
    DECLARE contador INT DEFAULT 1;
    DECLARE total INT DEFAULT 0;

    WHILE contador <= p_limite DO

        IF MOD(contador, 2) = 0 THEN
            SET total = total + contador;
        END IF;

        SET contador = contador + 1;

    END WHILE;

    SELECT total AS suma_numeros_pares;
END //

DELIMITER ;

CALL sumar_pares(10);
```

O procedemento percorre os números desde 1 ata o límite recibido como parámetro. En cada iteración comproba se o número é par e, nese caso, engádeo ao acumulador.

Para o valor 10, o resultado será a suma de 2 + 4 + 6 + 8 + 10.

Exemplo: calcular un factorial

```
DELIMITER //

CREATE PROCEDURE calcular_factorial(IN p_numero INT)
BEGIN
    DECLARE contador INT DEFAULT 1;
    DECLARE resultado BIGINT DEFAULT 1;

    IF p_numero < 0 THEN
        SELECT 'Non se admiten números negativos' AS mensaxe;
    ELSE
        WHILE contador <= p_numero DO
            SET resultado = resultado * contador;
            SET contador = contador + 1;
        END WHILE;

        SELECT resultado AS factorial;
    END IF;
END //

DELIMITER ;

CALL calcular_factorial(5);
```

O procedemento comproba primeiro que o número non sexa negativo. A continuación, utiliza un bucle para multiplicar os números desde 1 ata o valor recibido.

O factorial de 5 é 120.

7. Boas prácticas na programación procedemental

- Utilizar nomes de variables e etiquetas que describan claramente a súa finalidade.
- Declarar todas as variables locais ao comezo do bloque correspondente.
- Utilizar `IF` para condicións sinxelas e `CASE` cando existan varias alternativas.
- Escoller `WHILE` cando a condición de repetición deba comprobarse antes de executar o bloque.
- Utilizar `REPEAT` cando se necesite executar o bloque polo menos unha vez.
- Empregar `LOOP` cando se queira controlar explicitamente o momento de saída.
- Comprobar que todos os bucles teñen unha condición de saída.
- Actualizar correctamente as variables que controlan as iteracións.

- Utilizar `LEAVE` e `ITERATE` cando faciliten a comprensión do fluxo do programa.
- Indentar o código e separar os bloques para facilitar a súa lectura e mantemento.

8. Exercicios prácticos

Os seguintes exercicios permiten practicar a creación de procedementos almacenados que incorporan condicións, seleccións e estruturas repetitivas.

Exercicio 1. Condicionais IF

1. Crea un procedemento que reciba unha nota e indique se o alumno aprobou ou suspendeu.
2. Amplía o procedemento para clasificar as notas en varias categorías.
3. Crea un procedemento que reciba unha idade e determine se unha persoa é maior de idade.
4. Define un procedemento que reciba un prezo e indique se supera un límite establecido.
5. Utiliza condicións compostas con `AND` e `OR`.

Exercicio 2. Estrutura CASE

1. Crea un procedemento que reciba un número do 1 ao 7 e devolva o día da semana correspondente.
2. Define un procedemento que reciba un código de estado e mostre unha descrición.
3. Utiliza un CASE buscado para clasificar produtos segundo o seu prezo.
4. Engade unha alternativa por defecto para os valores non recoñecidos.
5. Compara a solución mediante CASE coa solución equivalente mediante IF.

Exercicio 3. Estruturas WHILE e REPEAT

1. Crea un procedemento que mostre os números comprendidos entre 1 e un límite recibido como parámetro.
2. Realiza unha versión utilizando WHILE e outra utilizando REPEAT.
3. Calcula a suma dos números comprendidos entre 1 e un límite.
4. Calcula a táboa de multiplicar dun número recibido como parámetro.
5. Comproba o comportamento dos dous bucles cando o límite inicial non permite realizar iteracións.

Exercicio 4. Estrutura LOOP e control de iteracións

1. Implementa un contador utilizando LOOP.
2. Utiliza LEAVE para finalizar o bucle cando se alcance un límite.
3. Utiliza ITERATE para saltar determinadas iteracións.

4. Mostra os números impares comprendidos entre dous valores.
5. Realiza un procedemento que acumule valores ata alcanzar unha cantidade determinada.

Exercicio 5. Combinación de estruturas

1. Crea un procedemento que calcule o factorial dun número.
2. Define un procedemento que sume os números pares ata un límite.
3. Elabora un procedemento que conte cantos números dunha serie son múltiplos dun valor recibido.
4. Utiliza condicións para validar os parámetros recibidos.
5. Combina IF, CASE e estruturas repetitivas nun mesmo procedemento.

Exercicio 6. Pequeno guión procedemental

Desenvolve un procedemento almacenado que realice unha operación de procesamento sobre unha táboa da base de datos utilizada nas prácticas.

O procedemento deberá:

- Recibir polo menos dous parámetros.
- Declarar e utilizar variables locais.
- Validar os datos recibidos mediante condicións.
- Utilizar unha estrutura de selección.
- Incorporar polo menos un bucle.
- Utilizar `LEAVE` ou `ITERATE`.
- Devolver información sobre o resultado da operación.

9. Resumo do bloque

Estrutura	Finalidade	Sintaxe principal
BEGIN ... END	Agrupar instrucións nun bloque.	`BEGIN ... END`
IF	Executar instrucións segundo unha condición.	`IF ... THEN ... ELSE ... END IF`
CASE	Seleccionar entre varias alternativas.	`CASE ... WHEN ... THEN ... END CASE`
WHILE	Repetir mentres se cumpra unha condición.	`WHILE ... DO ... END WHILE`
REPEAT	Repetir ata que se cumpra unha condición.	`REPEAT ... UNTIL ... END REPEAT`
LOOP	Crear un bucle con saída controlada.	`LOOP ... END LOOP`
LEAVE	Abandonar un bloque ou bucle.	`LEAVE etiqueta`
ITERATE	Pasar á seguinte iteración.	`ITERATE etiqueta`

> **Obxectivo alcanzado**
> Ao rematar este bloque, o alumnado debe ser capaz de construír pequenos programas procedementais en MariaDB, agrupando instrucións en bloques, tomando decisións mediante condicións e seleccións e repetindo operacións mediante estruturas de control de fluxo.